

تمرین چهارم

برنامه نویسی پیشرفته

دکتر ملکی مجد

۱۶ فروردین ۱۴۰۴

زمستان - بهار ۴۰۳۲



دانشگاه علم و صنعت ایران

نکات مهم تمرین

- تنها زبان برنامه‌نویسی مجاز برای این تمرین سی شارپ است.
- از پاسخ تحویل داده شده تمرین ها ارائه گرفته خواهد شد.
- در صورت مشاهده هرگونه کپی برداری هم کپی کننده و هم کپی شونده نمره صفر خواهند گرفت.
- در صورت مشاهده استفاده از هرگونه LLM برای پاسخ گویی به سوالات به جای شما، نمره شما صفر خواهد شد.
- برای هر سری تمرین سه روز مهلت تحویل با تاخیر دارید و مجموعاً ده روز مهلت ارسال با تاخیر دارید.

طراحان تمرین

در صورت بروز هرگونه مشکل یا سوالی در مورد تمرین، می‌توانید با طراحان تمرین تماس بگیرید.

• سوال اول - محمد یاوری @EmmWhy

• سعید نوریان - سوال دوم @Saeedvft

نکته مهم تمرین

بایستی مدیریت خطا در برنامه‌های خود را به خوبی انجام دهید. برای این کار باید از استثناء-ها استفاده کنید و همچنین باید پیغام‌های مناسبی برای کاربران برنامه‌تان نمایش دهید. در صورت عدم رعایت این نکته، نمره شما کاهش خواهد یافت.

۱ موزیک پلیر ساده

در این تمرین شما باید یک موزیک پلیر ساده را شبیه سازی کنید.

۱.۱ کلاس‌های مورد نیاز:

1.1.1 Song

دارای پراپرتی‌های:

• ID – Int (unique)

• Title – String

• Artist – Struct Artist

• Album – Struct Album

• Genre – String

• Duration – Timespan

• Year – Int سال انتشار

• CreatedAt – DateTime زمان ساخته شدن آهنگ توسط کاربر

• SongInfo – string همه اطلاعات آهنگ و دارای متد‌های:

• EditSongInfo(title, artistName, albumName, genre, duration, year)

هر کدام از ورودی‌ها که null یا مقدار دیفالت بودند ادیت نمی‌شوند و مقدار قبلی خود را حفظ می‌کنند. بقیه ورودی‌ها ادیت می‌شوند.

• Play()

نام آهنگ و روبه روی آن یک نوار کامل شونده چاپ می‌شود که به صورت زیر است:

[###_____]

[#####_____]

[#####_____]

...

پر شدن این نوار باید یک دهم مدت زمان آهنگ طول بکشد.

Playlist ۲.۱.۱

دارای پراپرتی های:

- (unique) Int – ID
- String – Name
- String - Description
- Song of List – Songs
- string – CommonGenre ژانری که بیشترین تکرار را در بین آهنگ ها دارد. در کانستراکتور ورودی داده نمی شود بلکه با اد شدن یا حذف شدن (یا ادیت شدن) هر آهنگ محاسبه و ست می شود.
- double – AvgDuration مانند پراپرتی بالایی
- Album of list – Albums
- DateTime – CreatedAt زمان ساخت پلی لیست توسط کاربر
- string – PlaylistInfo همه اطلاعات به جز اطلاعاتی که در متد های کلاس نمایش داده می شوند. دارای متد های:
 - FindSong(songID)
 - آهنگ با آیدی داده شده را بر می گرداند.
 - ListSongs()
 - اطلاعات هر آهنگ را با استفاده از پراپرتی مربوط از کلاس Song چاپ میکند یا بر میگرداند.
 - ListAlbums()
 - اطلاعات مربوط به آلبوم ها را چاپ می کند.
 - ListArtists()
 - اطلاعات مربوط به آرتیست ها را چاپ می کند.
 - AddSong(songID)
 - یک آهنگ از پیش ساخته شده را به پلی لیست اضافه می کند.
 - RemoveSong(songID)
 - یک آهنگ را از پلی لیست حذف می کند.
- FilterSongsBy(title, genre, durationMin, durationMax, artistName, albumName, yearMin, yearMax)
- فیلتر روی آهنگ ها بر اساس ورودی هایی که مقدار دیفالت ندارند اعمال می شود. (ممکن است لازم باشد چند شرط همزمان بررسی شوند)
- دقت کنید که لازم نیست مقدار ورودی برای title, genre, artistName, albumName دقیقا با مقدار آهنگی مطابقت داشته باشند که آن آهنگ نمایش داده شود و اگر با

بخشی از آن مقدار هم مطابقت داشته باشند آهنگ نمایش داده می شود (جستجو case-sensitive نیست).

Archive ۳.۱.۱

دارای پراپرتی های:

- User's name – string از فایل خوانده شود و در فایل به صورت دستی نوشته شود
- Song of List – Songs
- Artist of List – Artists
- Album of List – Albums
- Playlist of List – Playlists

دارای متد های:

- FindSong(songID) آهنگ با آیدی داده شده را بر می گرداند.
- FindPlaylist(playlistID) پلی لیست با آیدی داده شده را بر می گرداند.
- AddSong(...) با دریافت اطلاعات لازم یک آهنگ می سازد و آن را به آرشیو اضافه میکند.
- AddPlaylist(...) با دریافت اطلاعات لازم یک پلی لیست می سازد و آن را به آرشیو اضافه میکند. لیست آهنگ های پلی لیست ابتدا خالی است.
- RemoveSong(ID) آهنگ با آیدی مورد نظر را از آرشیو و پلی لیست ها حذف می کند.
- FilterSongsBy(title, genre, durationMin, durationMax, artistName, albumName, yearMin, yearMax) فیلتر روی آهنگ ها بر اساس ورودی هایی که مقدار دیفالت ندارند اعمال می شود. (ممکن است لازم باشد چند شرط همزمان بررسی شوند) دقت کنید که لازم نیست مقدار ورودی برای title, genre, artistName, albumName دقیقاً با مقدار آهنگی مطابقت داشته باشند که آن آهنگ نمایش داده شود و اگر با بخشی از آن مقدار هم مطابقت داشته باشند آهنگ نمایش داده می شود. جستجو case-sensitive نیست.
- ListSongs() مشابه متد کلاس پلی لیست
- ListAlbums() مشابه متد کلاس پلی لیست

- ListArtists()
- مشابه متد کلاس پلی لیست
- ListPlaylists()
- اطلاعات پلی لیست ها لیست شود.
- FilterAlbumsByName(name)
- آلبوم هایی با اسم داده شده نمایش داده شوند. دقت کنید که لازم نیست مقدار ورودی دقیقا با مقدار اسم آلبومی مطابقت داشته باشند که آن آلبوم نمایش داده شود و اگر با بخشی از آن مقدار هم مطابقت داشته باشند آهنگ نمایش داده می شود. جستجو case-sensitive نیست.
- FilterArtistsByName(name)
- آرتیست هایی با اسم داده شده نمایش داده شوند. دقت کنید که لازم نیست مقدار ورودی دقیقا با مقدار اسم آرتیست مطابقت داشته باشند که آن آرتیست نمایش داده شود و اگر با بخشی از آن مقدار هم مطابقت داشته باشند آهنگ نمایش داده می شود. جستجو case-sensitive نیست.
- Menu
- منو برای اجرای دستورات! طراحی منو به عهده خودتان است، به نحوی که همه متد ها توسط کاربر قابل استفاده باشند.

۲.۱ استراکت های لازم:

۱.۲.۱ Artist

- Int - ID
- string – Name
- String – Bio
- string – Genre
- list of string – NamesOfMembers
- string - ArtistInfo
- تنها لازم است کانستراکتور داشته باشد

۲.۲.۱ Album

- Int - ID
- string – Name
- string – Description
- string – Genre

- Int – songCount
- Int – ReleaseYear
- string – ArtistName
- string – AlbumInfo

تنها لازم است کانستراکتور داشته باشد

۳.۱ نکات:

- دقت کنید مدیریت خطا باید به صورت کامل صورت گیرد و برنامه شما تحت هیچ شرایطی کرش (Crash) نکند.
- کار با فایل برای ذخیره اطلاعات ضروری است و با بستن برنامه نباید داده ای از دست برود.
- برای ذخیره سازی ها می توانید از روش Json Serialization استفاده کنید اطلاعات بیشتر را می توانید از لینک زیر بخوانید: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>
- ID ها را خود مقدار داده و از ورودی نخوانید.
- برای چاپ مقادیر مربوط به استراکت ها از توابع getter آنها استفاده کنید.
- پراپرتی هایی که پسوند info دارند، تنها تابع getter دارند که اطلاعات گفته شده را بر می گردانند.
- اضافه کردن پراپرتی یا فیلد طبق تشخیص خودتان بلامانع است.

۲ مدیریت کارمندان

در این تمرین، شما مسئولیت طراحی و پیاده‌سازی سامانه‌ای برای مدیریت پیشرفته کارکنان را بر عهده دارید. سامانه‌ی شما باید امکان ذخیره‌سازی اطلاعات کارمندان، مدیریت کارمندان، تخصیص شیفت، تخصیص حقوق، بررسی امکان اضافه‌کاری، تخصیص پاداش و ... را به کاربران بدهد. این سامانه از استراکت‌ها، پراپرتی‌ها، Enums و منطق اعتبارسنجی استفاده می‌کند.

۱.۲ Enums مورد نیاز:

۱.۱.۲ نوع شیفت (ShiftType):

- Morning :۶am-۲pm
- Evening :۲pm-۱۰pm
- Night :۱۰pm-۶am
- Custom: ساعت دلخواه
- OnCall: آماده به کار اضطراری

۲.۱.۲ دپارتمان (Department):

- Engineering
- Sales
- HR
- Operations
- Finance
- Marketing

۳.۱.۲ وضعیت استخدام (EmploymentStatus):

- Active (فعال)
- OnLeave (در حال ترک سازمان)
- Terminated (غیرفعال)

• Probation (آزمایشی)

۴.۱.۲ سطح حقوق (PayGrade):

- Intern
- Junior
- MidLevel
- Senior
- Lead
- Manager

هر سطح حقوق (PayGrade) یک محدوده‌ی حقوق مشخص دارد که در دیکشنری PayGradeRanges تعریف می‌شود و شامل حداقل و حداکثر حقوق مجاز است. در هر جای برنامه که به Salary و Grade نیاز داشتیم (مثل سازنده، ترفیع یا افزایش حقوق)، از دیکشنری برای بررسی کردن محدوده استفاده می‌شود.

```
private static readonly Dictionary<PayGrade, (decimal Min, decimal Max)> PayGradeRanges = new()  
{  
    { PayGrade.Intern, (20000, 30000) },  
    { PayGrade.Junior, (30000, 50000) },  
    { PayGrade.MidLevel, (50000, 80000) },  
    { PayGrade.Senior, (80000, 120000) },  
    { PayGrade.Lead, (120000, 160000) },  
    { PayGrade.Manager, (160000, 250000) }  
};
```

۲.۲ استراکت‌ها

Time ۱.۲.۲

این استراکت شامل فیلدهای زیر است:

- Hour
- Minute

WorkShift ۲.۲.۲

این استراکت شامل فیلدهای زیر است:

- StartTime (Time)

• (Time) EndTime

• (ShiftType) ShiftType

• (int) OvertimeHours

• (int) DaysPerWeek

نکات مهم:

• تمامی فیلدها باید با پراپرتی‌ها کنترل شوند و فقط از طریق constructor مقداردهی شوند.

• شیفت OnCall نباید از ۱۲ ساعت بیشتر باشد، مگر اینکه IsOvertime برابر True باشد.

• اگر ShiftType برابر با Morning، Evening یا Night بود، مقادیر StartTime و EndTime نباید از کاربر دریافت شوند و به صورت خودکار مقداردهی شوند.

متدهای مورد نیاز:

• IsValidShift: بررسی کند که آیا شیفت معتبر است یا خیر (بر اساس قوانین بالا) و bool برگرداند.

• GetShiftHours: طول شیفت (+ طول اضافه‌کاری) را به ساعت محاسبه کند و برگرداند (double).

• AddOvertime: ساعت ورودی بگیرد و مقدار آن را به OvertimeHours اضافه کند.

۳.۲ کلاس Employee

این کلاس به مدیریت اطلاعات کارکنان و شیفت‌ها می‌پردازد. فیلدهای این کلاس به صورت زیر است:

• EmployeeId (int): شناسه یکتا (فقط خواندنی پس از مقداردهی).

• Name (string): نام کارمند.

• Grade (PayGrade): سطح کارمند.

• Salary (decimal): حقوق.

- Department (Department): دپارتمان.
- Status (EmploymentStatus): وضعیت استخدام.
- Shift (WorkShift): شیفت.
- HireDate (DateTime): تاریخ استخدام (فقط خواندنی).
- PerformanceRating (double): امتیاز عملکرد (۱ تا ۵).
- YearsOfService (int): میزان سابقه‌ی کاری شخص که برابر با اختلاف زمان فعلی و HireDate است.

نکات مهم:

- از پراپرتی‌ها برای مقداردهی تمامی فیلدها استفاده شود.
- با استفاده از setter اطمینان حاصل کنید که Name نمی‌تواند null یا خالی باشد و HourlySalary مقدار مثبت دارد.
- مقدار Salary باید در محدوده‌ی تعریف‌شده در PayGradeRanges برای سطح کارمند (Grade) باشد. در غیر این صورت پیغام خطای مناسب چاپ شود.

متدهای مورد نیاز:

- ApplyRaise: درصد افزایش حقوق را به عنوان ورودی می‌گیرد و حقوق شخص مدنظر را افزایش می‌دهد.
- CalculateBonus: پاداش هر کارمند را محاسبه می‌کند و برمی‌گرداند.
- CanTakeOvertime: بررسی می‌کند که آیا کارمند امکان استفاده از اضافه‌کاری‌های خود را دارد یا خیر.
- PromoteTo: درجه‌ی شخص را ترفیع می‌بخشد.
- AddOvertime: به شیفت یک کارمند بخصوص، تایم اضافه‌کاری تخصیص می‌دهد.

۴.۲ منوی برنامه

برنامه در ابتدا منویی با گزینه‌های زیر نمایش می‌دهد:

۱. AddEmployee: این گزینه به عنوان ورودی آیدی کارمند را دریافت کرده و در صورت موجود نبودن کارمند با آن آیدی، اطلاعات تکمیلی کارمند را دریافت می‌کند.
۲. ShowInformation: نمایش اطلاعات کارمند.
۳. ApplyRaise: شناسه کارمند و درصد افزایش از کاربر دریافت می‌شود.
۴. CalculateBonus: شناسه کارمند دریافت می‌شود.
۵. CanTakeOvertime: شناسه کارمند دریافت می‌شود.
۶. Promote: شناسه کارمند و سطح جدید از کاربر دریافت می‌شود.
۷. Exit: خروج از برنامه.

نکات مهم:

- بعد از افزودن هر کارمند، اطلاعات مربوط به آن را در فایل Employees.json ذخیره کنید.
- پس از هر بار آپدیت کارمند، فایل مذکور را ویرایش و آپدیت کنید.